

HYDRA

[Kaynak](#)

İÇERİK

1. Hydra: Çevrimiçi Parola Kırma ve Brute Force Temelleri
2. Hydra: Hedef Sisteme Bağlantı ve İleri Seviye Brute Force Operasyonları
3. Hydra ile İleri Seviye Brute Force ve Web Kimlik Doğrulama Analizi
4. Hydra: SSH Servisine Yönelik İleri Seviye Brute Force ve Sistem İçeri Enumeration

Hydra: Çevrimiçi Parola Kırma ve Brute Force Temelleri

Hydra'nın İşlevi ve Arka Plan Mantığı

Hydra, ağ üzerinden kimlik doğrulaması gerektiren servislere yönelik otomatikleştirilmiş bir çevrimiçi parola kırma aracıdır. Temel amacı, bir hedefin giriş mekanizmasına yönelik hızlı ve sistematik bir **Brute Force** (Kaba Kuvvet Saldırısı: Hedef sistemdeki geçerli kullanıcı adı ve parola kombinasyonunu bulana kadar, geniş bir sözlük dosyasındaki tüm olasılıkların otomatik olarak denenmesi işlemi) gerçekleştirmektir.

Manuel olarak bir servisin (örneğin **SSH**, **FTP**, **SNMP** veya bir web uygulamasının giriş formu) parolasını tahmin etmeye çalışmak pratik ve uygulanabilir bir yöntem değildir. **Hydra**, bu süreci bir **Wordlist** (Kelime Listesi: Sızma testlerinde kullanılan, sızdırılmış veya yaygın olarak tercih edilen milyonlarca parolanın bulunduğu devasa metin dosyası) aracılığıyla otomatize eder. Araç, ağ protokolünün doğasını taklit ederek saniyeler içinde binlerce giriş denemesi yapar ve doğru kimlik bilgilerini tespit edene kadar işlemi sürdürür.

Genişletilmiş Hedef Yüzeyi ve Protokol Desteği

Hydra'nın sektördeki en güçlü çevrimiçi parola kırma araçlarından biri olmasının temel nedeni, kimlik doğrulama mekanizmalarına sahip neredeyse tüm modern ve eski ağ protokolleriyle entegre çalışabilmesidir. Resmi deposuna göre araç, arka planda ilgili protokolün el sıkışma (handshake) ve oturum açma (login) standartlarını birebir uygulayarak aşağıdaki servisleri hedef alabilir:

Asterisk, AFP, Cisco AAA, Cisco auth, Cisco enable, CVS, Firebird, **FTP**, HTTP-FORM-GET, HTTP-FORM-POST, HTTP-GET, HTTP-HEAD, HTTP-POST, HTTP-PROXY, HTTPS-FORM-GET, HTTPS-FORM-POST, HTTPS-GET, HTTPS-HEAD, HTTPS-POST, HTTP-Proxy, ICQ, **IMAP**, IRC, **LDAP**, MEMCACHED, MONGODB, MS-SQL, **MYSQL**, NCP, NNTP, Oracle Listener, Oracle SID, Oracle, PC-Anywhere, PCNFS, **POP3**, POSTGRES, Radmin, **RDP**, Rexec, Rlogin, Rsh, RTSP, SAP/R3, SIP, **SMB**, **SMTP**, SMTP Enum, SNMP v1+v2+v3,

SOCKS5, **SSH** (v1 ve v2), SSHKEY, Subversion, TeamSpeak (TS2), Telnet, VMware-Auth, VNC ve XMPP.

Bu geniş yelpaze, bir sızma testi (penetration testing) sırasında ağda keşfedilen herhangi bir açık portun potansiyel bir saldırı vektörü olabileceğini gösterir. Ayrıntılı protokol yapılandırmaları ve modül parametreleri için Kali Linux'un yerleşik **Hydra** dokümantasyonu her zaman nihai referans noktasıdır.

Zafiyetin Temel Nedeni: Zayıf Parolalar ve Varsayılan Yapılandırmalar

Hydra gibi araçların başarılı olmasının temelinde sistemin teknik bir açığı değil, insan faktörü ve yapılandırma hataları yatar. Güçlü bir parolanın önemi bu noktada ortaya çıkar. Eğer bir parola yaygın kullanıma sahipse, özel karakterler (% , & , ! , vb.) içermiyorsa ve sekiz karakterden kısaysa, standart bir sözlük saldırısına (dictionary attack) karşı tamamen savunmasızdır.

Saldırganlar, içerisinde yüz milyonlarca yaygın parolanın bulunduğu (örneğin popüler `rockyou.txt`) listeler kullanırlar. Bu nedenle, bir uygulama veya donanım (özellikle Kapalı Devre Televizyon - CCTV kameraları veya hazır web framework'leri) kutudan çıktığı haliyle ağa dahil edildiğinde büyük bir risk oluşturur.

ÖNEMLİ: Birçok üretici, cihazların ilk kurulumunu kolaylaştırmak için aşağıdaki gibi varsayılan kimlik bilgileri atar:

```
Kullanıcı Adı: admin  
Parola: password
```

Bu varsayılan kimlik bilgileri (Default Credentials), saldırganların ağdaki **Enumeration** (Bilgi Toplama: Hedef sistemdeki açık portlar, servis versiyonları ve yapılandırma detaylarını haritalandırma süreci) aşamasında aradıkları ilk zafiyettir. Varsayılan parolaların kurulum anında değiştirilmemesi, ağın en dış çevresindeki güvenliğin (perimeter security) saniyeler içinde aşılmasına ve saldırganın sistemde yetkili bir oturum elde etmesine (Initial Access) doğrudan yol açar. Sistemin kendi mimarisi ne kadar güvenli olursa olsun, zayıf bir kapı kilidi tüm savunmayı anlamsız kılar.

Hydra: Hedef Sisteme Bağlantı ve İleri Seviye Brute Force Operasyonları

Ortam Hazırlığı ve Kurulum (Installation & Environment Setup)

Sızma testlerinde kullanılan standart işletim sistemlerinde (örneğin Kali Linux veya TryHackMe platformunun sunduğu AttackBox), **Hydra** genellikle önceden kurulmuş olarak gelir. Ancak bağımsız bir sistemde çalışıyorsanız veya farklı bir Linux dağıtımı kullanıyorsanız, aracı resmi depolarından (repositories) doğrudan çekmeniz gerekir.

Ubuntu veya Debian tabanlı bir sistemde **Hydra** kurulumunu başlatmak için aşağıdaki komutu kullanırız:

```
apt install hydra
```

Eğer Fedora veya RedHat tabanlı bir sistem üzerinde çalışıyorsanız, paket yöneticisi farklılaştığı için kurulumu şu şekilde gerçekleştiririz:

```
dnf install hydra
```

Bu komutlar, aracın gerekli bağımlılıklarıyla (dependencies) birlikte sisteme entegre olmasını sağlar. Operasyon sırasında hedef makine olan **10.113.143.146** IP adresine yönelik saldırılar gerçekleştireceğiz.

Hydra Sözdizimi (Syntax) ve Temel Servislere Yönelik Saldırılar

Hydra'ya geçeceğimiz argümanlar (flags/parameters), doğrudan hedeflediğimiz servise (protokole) bağlıdır. Aracın çalışma mantığı, belirtilen protokole uygun bir ağ isteği oluşturmak ve bunu yapılandırdığımız sözlük dosyasıyla (Wordlist) otomatize etmektir.

Örneğin, hedef sistemde çalışan bir **FTP** (File Transfer Protocol - Dosya Aktarım Protokolü) servisine yönelik bir saldırı vektörü oluşturmak istediğimizi varsayalım:

```
hydra -l user -P passlist.txt ftp://10.113.143.146
```

Bu komutta, belirli bir kullanıcı adı (`user`) bildiğimizi varsayarak parolaları `passlist.txt` üzerinden deniyoruz ve hedef servis olarak **ftp** protokolünü işaret ediyoruz.

SSH (Secure Shell) Brute Force Operasyonu

Sızma testlerinde en sık karşılaşılan senaryolardan biri, uzaktan yönetim için yapılandırılmış ancak zayıf parolalara sahip **SSH** servislerini hedef almaktır. Hedef makinedeki (10.113.143.146) SSH servisine yönelik optimize edilmiş bir saldırı komutu şu şekildedir:

```
hydra -l root -P passwords.txt 10.113.143.146 -t 4 ssh
```

Bu komutun arka plandaki teknik anlamını ve parametrelerini detaylandıralım:

- `-l root` : Bu argüman, hedeflenen tekil kullanıcı adını (Login) belirtir. (Eğer tek bir kullanıcı adı değil de, birden fazla kullanıcı adını bir listeden denemek isteseydik küçük `-l` yerine büyük `-L users.txt` argümanını kullanmamız gerekirdi). Burada doğrudan **root** kullanıcıasını hedefliyoruz çünkü bu hesaba erişim, sistemde **Privilege Escalation** (Hak Yükseltme: Düşük yetkili bir hesaptan, sistem üzerinde tam kontrol sağlayan root/administrator seviyesine geçiş yapma) aşamasını tamamen atlamamızı sağlar.

- `-P passwords.txt` : Parola listesinin (Wordlist) yolunu belirtir. Eğer sadece tek bir parolayı test etmek isteseydik küçük `-p password123` kullanırdık. Büyük harf her zaman dosya okutmak anlamına gelir.
- `10.113.143.146` : Hedefin IP adresidir.
- `-t 4` : Aynı anda açılacak paralel işlem parçacığı (Threads) sayısını belirler. Bu parametre kritik öneme sahiptir.
- `ssh` : Hydra'ya bağlantıyı hangi protokol standartlarına göre kuracağını söyler.

ÖNEMLİ: Neden `-t 4` kullanıyoruz? İşlem sayısını `-t 64` yaparak süreci aşırı hızlandırmak mantıklı görünse de, hedef sistemin kaynakları veya güvenlik duvarları (Firewall/IDS) saniyede yüzlerce isteğe yanıt veremeyebilir. Bu durum ağda darboğaz yaratır, servisin çökmesine (Denial of Service) neden olur veya hedef sunucu IP adresimizi bloke eder. Güvenilir ve tutarlı sonuçlar almak için genellikle thread sayısını SSH gibi hassas servislerde 4 ile 16 arasında tutmak idealdir.

Web Formlarına Yönelik (HTTP-POST-FORM) Brute Force

Web uygulamalarındaki giriş sayfaları (Login Forms), doğrudan bir ağ protokolü üzerinden değil, **HTTP** protokolünün GET veya POST metodları üzerinden çalışır. **Hydra**, web tabanlı kimlik doğrulama mekanizmalarını aşmak için özel modüllere sahiptir. Ancak bu saldırıyı başlatmadan önce, formun nasıl çalıştığını analiz etmemiz gerekir. Tarayıcıların "Geliştirici Araçları" (Developer Tools) altındaki "Ağ" (Network) sekmesini kullanarak giriş isteğini yakalamalı ve hangi parametrelerin (örneğin `user`, `username`, `email` vb.) sunucuya gönderildiğini tespit etmeliyiz.

Standart bir POST metodlu giriş formuna yönelik gelişmiş **Hydra** komutu şu formattadır:

```
hydra -l <username> -P <wordlist> 10.113.143.146 http-post-form  
"/:username=^USER^&password=^PASS^:F=incorrect" -V
```

Bu karmaşık görünen modül dizisinin (syntax) arka plandaki çalışma mantığı şu şekildedir:

1. `http-post-form` : Hydra'ya hedefin bir web formu olduğunu ve verilerin HTTP POST isteğinin gövdesinde (body) gönderilmesi gerektiğini söyler.
2. Tırnak içindeki bölüm `"URL:PARAMETRELER:HATA_MESAJI"` olmak üzere iki nokta (:) ile ayrılmış üç ana parçadan oluşur.
3. **Birinci Parça (/)**: Giriş işleminin yapıldığı yol (Path). Eğer giriş sayfası `http://10.113.143.146/login.php` olsaydı, buraya `/login.php` yazardık. Burada doğrudan ana dizine (/) istek atıyoruz.
4. **İkinci Parça (username=^USER^&password=^PASS^)**: Ağ sekmesinden yakaladığımız, sunucuya gönderilen ham POST verisi (Payload). **Hydra**, `-l` parametresinden aldığı değeri `^USER^` değişkeninin yerine, `-P` parametresinden okuduğu listeyi ise sırayla `^PASS^` değişkeninin yerine yerleştirerek (inject ederek) istekleri otomatik olarak sunucuya yollar.

5. **Üçüncü Parça (F=incorrect)**: Bu, saldırının başarıya ulaşip ulaşmadığını belirleyen **en kritik kısımdır**. Web sunucuları genellikle başarısız girişlerde de "200 OK" HTTP durum kodu döndürürler. Bu nedenle Hydra, sadece sayfa koduna bakarak başarılı olup olmadığını anlayamaz. Bizim, başarısız bir girişte ekrana yazdırılan spesifik bir hata metnini (örneğin "incorrect", "Invalid login", "Kullanıcı bulunamadı") araca belirtmemiz gerekir. F= (Fail - Başarısızlık) parametresi araca şunu söyler: "Eğer dönen HTTP yanıtının içinde 'incorrect' kelimesi geçiyorsa, denenen parola yanlıştır, bir sonrakine geç. Eğer bu kelimeyi görmezsen, demek ki başarılı oldun ve oturum açtın!"
6. **-v (Verbose)**: Arka planda denenen her bir kullanıcı adı/parola kombinasyonunu terminale anlık olarak (canlı) basar. Bu, saldırının doğru gidip gitmediğini, takılıp takılmadığını izlemek (debugging) için şarttır.

DİKKAT (Alternatif Senaryo): Eğer hedef web sunucusu standart HTTP portu olan 80 veya HTTPS portu olan 443 yerine farklı bir portta (örneğin 8080) çalışıyorsa, komut başarısız olacaktır. Hedefi ağacın farklı bir dalında aramak için `-s <port>` parametresini ekleyerek rotayı düzeltmemiz gerekir:

```
hydra -l <username> -P <wordlist> 10.113.143.146 http-post-form  
"/:username=^USER^&password=^PASS^:F=incorrect" -s 8080 -v
```

Bu sayede Hydra, standart dışı yapılandırmalara sahip sistemlerde dahi tam isabetli hedefleme yapabilir. Başarısızlık senaryolarında her zaman hedef portun doğruluğunu, ağ trafiğinin yapısını ve hedef kelimeyi (invalid response string) tekrar teyit etmek zorundayız.

Hydra ile İleri Seviye Brute Force ve Web Kimlik Doğrulama Analizi

Hydra'nın İşlevi ve Operasyonel Arka Planı

Hydra, ağ üzerinden kimlik doğrulaması (authentication) gerektiren servislere yönelik otomatikleştirilmiş bir çevrimiçi **Brute Force** (Kaba Kuvvet) aracıdır. Sistemlerin giriş mekanizmalarına yönelik geniş bir **Wordlist** (Sözlük dosyası) kullanarak saniyeler içinde binlerce geçerli/geçersiz kombinasyonu dener.

Aracın sızma testlerindeki (penetration testing) birincil gücü, **SSH, FTP, RDP, LDAP, SMB, MySQL** ve web formları (HTTP-POST/GET) dahil olmak üzere 50'den fazla ağ protokolünün el sıkışma (handshake) ve oturum açma standartlarını arka planda simüle edebilmesidir.

Hydra gibi araçların yüksek başarı oranına sahip olmasının temel nedeni sistemlerin teknik zafiyetleri değil, insan faktörü ve yapılandırma hatalarıdır. Cihazların (özellikle CCTV'ler, router'lar veya hazır web framework'leri) kutudan çıktığı haliyle, varsayılan kimlik bilgileriyle (admin:password vb.) bırakılması, ağın dış çevresindeki güvenliğin anında çökmesine ve saldırganın **Initial Access** (İlk Erişim) elde etmesine neden olur.

Ağ Protokollerine Yönelik Saldırılar: SSH Senaryosu

Hedef sistemde (örneğin **10.113.143.146**) yapılandırılmış ancak zayıf parolaya sahip bir SSH servisine yönelik standart komut yapısı şöyledir:

```
hydra -l root -P passwords.txt 10.113.143.146 -t 4 ssh
```

- `-l root` : Hedeflenen tekil kullanıcı adı. (Birden fazla kullanıcı denenecekse büyük `-L users.txt` kullanılır). **root** kullanıcısını hedeflemek, başarılı olduğunda **Privilege Escalation** (Hak Yükseltme) aşamasını atlayarak sistemde tam yetki sağlar.
- `-P passwords.txt` : Parola listesinin (Wordlist) yolu. Tek bir parola denenecekse küçük `-p` kullanılır.
- `-t 4` : Aynı anda açılacak paralel işlem (Thread) sayısı.
- `ssh` : Bağlantı kurulacak protokol.

DİKKAT: İşlem sayısını (`-t`) çok yüksek tutmak (örneğin 64) teoride saldırıyı hızlandırırsa da, pratikte hedef servisin kaynaklarını tüketerek **Denial of Service (DoS)** durumuna yol açabilir veya güvenlik duvarları (IDS/IPS) tarafından anında bloke edilmenize neden olur. Kararlılık ve gizlilik için 4-16 arası thread idealdir.

Kabuk (Shell) Yorumlama Hataları ve Sözdizimi (Syntax)

Siber güvenlik dokümantasyonlarında yer alan `<username>` veya `<wordlist>` gibi ifadeler **Placeholder** (Yer Tutucu) olup, doğrudan komut satırına yazılmamalıdır. Eğer bu ifadeler doğrudan terminale girilirse Bash Shell (Linux komut satırı kabuğu), `<` ve `>` işaretlerini **Input/Output Redirection** (Girdi/Çıktı Yönlendirme) operatörü olarak algılar. Hydra'ya parametre geçmek yerine o isimde bir dosya arayıp içeriğini yönlendirmeye çalışır. Bu mantıksız yapı, terminalde doğrudan `bash: syntax error near unexpected token '<'` hatasına neden olur. Komutlar her zaman gerçek ve spesifik verilerle doldurulmalıdır.

Web Formlarına Yönelik (HTTP-POST-FORM) Brute Force ve Yapısı

Web uygulamalarındaki giriş sayfaları doğrudan bir port/servis üzerinden değil, HTTP POST metodu üzerinden çalışır. Web tabanlı kimlik doğrulama mekanizmalarını kırmak için **Hydra**'nın özel modülü kullanılır. Modülün temel yapısı tırnak içinde ve iki nokta (:) ile ayrılmış üç parçadan oluşur: "URL_YOLU:POST_VERİSİ:HATA_MESAJI"

```
hydra -l molly -P /usr/share/wordlists/rockyou.txt 10.113.143.146 http-post-form "/login:username=^USER^&password=^PASS^:F=incorrect" -V
```

- `http-post-form` : Hydra'ya hedef hedefin bir web formu olduğunu belirtir.
- `/login` : İsteklerin yapılacağı dizin (Endpoint/Path).
- `username=^USER^&password=^PASS^` : Sunucuya gönderilen ham veri (Payload). Hydra, `-l` argümanını `^USER^` yerine, `-P` argümanını `^PASS^` yerine enjekte eder.
- `F=incorrect` : Fail (Başarısızlık) koşulu. Sunucu başarısız girişlerde HTTP 200 OK döndürebileceği için, Hydra'nın yanlış parolayı anlaması için sayfa kaynağındaki spesifik

hata metnini (örneğin "incorrect") bilmesi gerekir. Bu metin bulunursa parola yanlış demektir, bulunmazsa başarılı giriş yapılmıştır.

- -v : Verbose (Detaylı) çıktı modu. Süreci anlık izlemeyi sağlar.

False Negative (Yalancı Negatif) Analizi ve Ağ Trafiği Yakalama

Eğer Hydra kısa bir listede veya RockYou.txt'nin ilk satırlarında durması gerekirken yüzlerce parola denemeye devam ediyorsa (örneğin 800. parolayı geçiyorsa), işlem **False Negative** (Yalancı Negatif) durumuna düşmüş, yani araç doğru parolayı kör bir şekilde atlamış demektir. Bunun iki temel nedeni vardır: Hatalı POST parametre isimleri veya yanlış hedef dizin (Path).

Bunu çözmek için saldırı derhal durdurulmalı ve tarayıcının **Geliştirici Araçları (Network sekmesi)** veya **Burp Suite** kullanılarak bir kukla istek (dummy request) gönderilmeli ve trafik yakalanmalıdır (Intercept).

Yakalanan ham HTTP POST isteğinin incelenmesi kritik istihbarat sağlar:

1. **Request-Line Analizi:** POST /login HTTP/1.1 satırı, uygulamanın kimlik doğrulama işlemlerini ana dizinde (/) değil, /login uç noktasında yaptığını kanıtlar. Hydra ana dizine istek attığında sunucu bu veriyi işlemez ve parola doğrulaması hiç gerçekleşmez.
2. **Request Body Analizi:** HTTP başlıklarının altındaki boşluktan (CRLF) sonra gelen username=user&password=pass verisi, formun arka planda beklediği değişken isimlerinin tam olarak username ve password olduğunu kesinleştirir.

Nihai İstismar (Exploitation) ve Başarılı Tespiti

Ağ trafiğinden elde edilen istihbaratla (doğru uç nokta olan /login dizininin komuta entegre edilmesi) güncellenen Hydra komutu, doğru hedefi vurmaya başlar.

```
[80][http-post-form] host: 10.113.143.146 login: molly password:
sunshine
1 of 1 target successfully completed, 1 valid password found
```

Hedef dizinin doğru yapılandırılması sayesinde sunucu POST verilerini doğru şekilde işler ve Hydra hata string'i eşleşmesini kusursuz bir şekilde analiz eder. Komut çalıştırıldıktan kısa bir süre sonra **molly** kullanıcısının geçerli parolasının **sunshine** olduğu tespit edilir ve ağın ilk savunma hattı (Authentication) başarıyla atlatılır.

Hydra: SSH Servisine Yönelik İleri Seviye Brute Force ve Sistem İçi Enumeration

Saldırı Vektörünün Değişimi: Neden SSH Hedefleniyor?

Bir önceki aşamada hedefin web paneline (/login) yönelik başarılı bir saldırı gerçekleştirmiştik. Ancak siber güvenlik operasyonlarında nihai hedef genellikle web

arayüzünde sınırlı kalmak değil, doğrudan işletim sistemi seviyesinde bir **Shell** (Kabuk: İşletim sistemine komut göndermemizi sağlayan komut satırı arayüzü) elde etmektir.

Görev bizden "molly" kullanıcısının SSH parolasını kırmamızı istiyor. Bu durum, web uygulamasında bulduğumuz parolanın (sunshine) SSH servisinde geçerli olmadığı (**Credential Reuse** zafiyetinin bulunmadığı) veya operasyonel olarak farklı bir savunma katmanının test edildiği anlamına gelir. Hedefimiz, **10.113.143.146** IP adresindeki **22. Portta** çalışan SSH servisini, **Hydra**'nın ağ protokolü yeteneklerini kullanarak istismar etmektir (Exploitation).

SSH Brute Force Komutunun İnşası ve Mantıksal Analizi

Hedef makinedeki SSH servisine yönelik optimize edilmiş kaba kuvvet (brute force) saldırısını başlatmak için aşağıdaki master komutu terminalde çalıştırıyoruz:

```
hydra -l molly -P /usr/share/wordlists/rockyou.txt 10.113.143.146 -t 4 ssh -V
```

Bu komutun teknik derinliği ve arka plandaki çalışma prensipleri şunlardır:

- **-l molly** : Oturum açma denemeleri için hedef kullanıcı adını (Login) "molly" olarak sabitler.
- **-P /usr/share/wordlists/rockyou.txt** : Daha önce sıkıştırmasını açtığımız (.gz formundan kurtardığımız) genişletilmiş parola listesinin (Wordlist) mutlak yoludur.
- **10.113.143.146** : Hedef sunucunun IP adresidir.
- **-t 4** : Aynı anda açılacak paralel işlem (Thread) sayısını 4 ile sınırlandırır.
- **ssh** : Hydra'nın hedef sistemle iletişim kurarken hangi ağ protokolünün kurallarını (handshake ve authentication mekanizmalarını) kullanacağını belirler.
- **-V** : (Verbose) Denenen parolaları anlık olarak ekrana basarak saldırı sürecinin körleşmesini engeller.

ÖNEMLİ: Neden web formlarında **-t** parametresi belirtmezken SSH için özellikle **-t 4** kullanıyoruz? SSH protokolü, doğası gereği yüksek eşzamanlı bağlantılara (concurrent connections) karşı son derece hassastır. Yüksek thread sayıları kullanmak, sunucunun **Rate Limiting** (İstek Sınırlandırma) veya **Fail2Ban** (Başarısız girişleri tespit edip IP'yi banlayan güvenlik mekanizması) savunmalarını anında tetikler. Bağlantılar düşer (Connection reset by peer hatası alınır) ve doğru parola denense bile SSH servisi yanıt vermeyeceği için **False Negative** (Yalancı Negatif) durumuna düşülür. SSH saldırılarında hızdan ziyade kararlılık esastır.

İstismar Sonucu ve Çıktı Analizi

Komut çalıştırıldıktan sonra arka planda yüzlerce kriptografik el sıkışma (SSH Handshake) gerçekleşecek ve doğru parola bulunduğunda Hydra süreci sonlandırarak aşağıdaki gibi bir çıktı üretecektir:


```
[22][ssh] host: 10.113.143.146 login: molly password: *****  
1 of 1 target successfully completed, 1 valid password found
```

- [22][ssh] : Hedefin 22 numaralı portunda çalışan SSH servisiyle başarılı bir iletişim kurulduğunu doğrular.
- ``login: molly password: *****``: Sistem, **rockyou.txt** içerisindeki "*****" parolasının SSH kimlik doğrulama mekanizmasını başarıyla atlattığını netleştirir. Artık sistemde **Initial Access** (İlk Erişim) sağlamak için gerekli anahtara sahibiz.

Post-Exploitation: Sisteme Giriş ve Bayrak (Flag) Tespiti

Elde ettiğimiz geçerli kimlik bilgileriyle hedef sistemde bir terminal oturumu açıyoruz.

```
ssh molly@10.113.143.146
```

Bağlantı kurulduğunda sunucu parolayı soracak, parolayı yazıp onayladıktan sonra yetkili bir kullanıcı olarak hedefin içine düşmüş olacağız. Artık kendi Kali makinemizde değil, kurbanın işletim sistemindeyiz.

Görev bizden "Flag 2" değerini bulmamızı istiyor. Sızma testlerinde (CTF etkinliklerinde) bayraklar genellikle gizli metin dosyaları olarak sistemin çeşitli yerlerine dağıtılır. Bayrağı bulmak için **Enumeration** (Bilgi Toplama/Keşif) komutlarını kullanmalıyız.

Eğer bayrak doğrudan kullanıcının ana dizininde (home directory) değilse, sistemin tamamında hedef odaklı bir arama yapmak için aşağıdaki gelişmiş Linux komutunu kullanırız:

```
find / -name "*flag2*" -type f 2>/dev/null
```

Bu komutun analitik parçalanması:

- `find /` : Aramayı işletim sisteminin kök dizininden (/) yani en tepeden başlatır. Tüm sistemi tarar.
- `-name "*flag2*"` : İsmnin içinde herhangi bir yerinde "flag2" geçen dosyaları arar. Yıldız işaretleri (*) wildcard (joker karakter) görevi görür.
- `-type f` : Arama sonuçlarını sadece dosyalar (file) ile sınırlandırır, aynı isimdeki klasörleri çöpe atar.
- `2>/dev/null` : (Hata Yönlendirmesi). Normal yetkilerle (molly) tüm sistemi taradığımız için, okuma iznimiz olmayan dizinlerde sürekli "Permission denied" (Erişim reddedildi) hataları fırlatılacaktır. Bu parametre, 2 numaralı standart hata çıktılarını (stderr) Linux'un kara deliği olan /dev/null konumuna yönlendirir. Böylece ekranda sadece bulduğumuz bayrağın temiz ve net dosya yolunu görürüz.

Komut bize dosyanın yerini (örneğin `/home/molly/Documents/flag2.txt`) verdiğinde, tek yapmamız gereken dosyanın içeriğini okumaktır:

```
cat /home/molly/Documents/flag2.txt
```

Bu komutun çıktısı, sertifika/eğitim görevinde senden istenen "Flag 2" değerinin kendisi olacaktır. Başarılı bir Enumeration ve Exploitation döngüsü bu şekilde kusursuzca tamamlanır.